

General

Running the code

To run the code, after compiling (see [Setup](#)), use the command:

```
trex-fitter <action(s)> <config file> [<options>]
```

The configuration file (`<config file>`) is a text file containing all the information on the definition of samples and fit regions, including all the fit and draw options. By default, the file `config/myFit.config` is loaded. See the section [Config File](#) for more details. Take a look at `config/myFit.config` or `config/ttH2015.config` to see some example config files. Most of the time, the only file the user has to modify to obtain their desired results is the configuration file.

The only mandatory argument, `<action(s)>` , tells TRExFitter which operation(s) to perform. The possible operations are defined in the main file (e.g. `util/trex-fitter.cc`). The available actions are:

Option	Action
<code>u</code>	read efficiencies, migration/response matrices and acceptances for unfolding and then fold them
<code>h</code>	read input histograms (valid only if the proper option is specified in the config file)
<code>n</code>	read input ntuples (valid only if the proper option is specified in the config file)
<code>w</code>	create the RooStats xmls and workspace
<code>f</code>	fit the workspace
<code>l</code>	calculate exclusion limit
<code>s</code>	calculate significance

Option	Action
<code>d</code>	draw pre-fit plots
<code>p</code>	draw post-fit plots
<code>a</code>	draw separation plots
<code>r</code>	draw ranking plot (see Ranking Plot)
<code>b</code>	re-run smoothing (in the future also rebinning)
<code>m</code>	multi-fit (see Multi-Fit)
<code>i</code>	grouped impact evaluation (see Grouped Impact)
<code>x</code>	run likelihood scan only, will not produce the standard fit output like pulls/correlation matrix/etc (useful with "LHscan" command line option for parallelization)
<code>u</code>	prepare input for the unfolding fit
<code>q</code>	run only the plotting part for the unfolding fit (useful when you want to replace the fit results ROOT file e.g. when using toys)
<code>t</code>	prepare inputs for templates (after <code>n</code> or <code>h</code> steps) needed for the template with reparametrised shape factors
<code>j</code>	translate the RooFit workspace into the HS3 format

New optional argument: `<options>`. It is a string (so make sure to use " or ' to enclose the string if you use more than one option) defining a list of options, in the form:

```
"<option1>=<value1>,<value2>,...:<option2>=..."
```

For details of available optional arguments, see the section [Command line options](#) below.

For details on how to run unfolding, see the [tutorial on Unfolding](#)

For details on how to run an EFT fit, see the [tutorial on EFT fits](#)

For details on how to compare or combine fits, see the [section on Multifits](#)

Config File

This section outlines the structure of the configuration file. A list of the available settings for the config file can be found in [this section](#).

The structure of the file should be the following:

```
<ObjectType>: <ObjectName>
  <ObjectProperty>: <Value>
  <ObjectProperty>: <Value>
  ...

<ObjectType>: <ObjectName>
  <ObjectProperty>: <Value>
  <ObjectProperty>: <Value>
  ...

...
```

NB: note the **blank** line between the objects!

The file should contain: * exactly one object of type `Job` * exactly one object of type `Fit` * exactly one object of type `Limit` * at least one object of type `Sample` * at least one object of type `Region` * any number of objects of type `Systematic` (even 0 is ok) * any number of objects of type `NormFactor` (even 0 is ok)

In case of unfolding you need: * exactly one object of type `Job` * exactly one object of type `Unfolding` * at least one object of type `TruthSample` * at least one object of type `UnfoldingSample`

Note that each object should have unique `<ObjectName>` .

At the beginning of TRExFitter execution, the config file used will be checked against a reference file. The reference files for single and multi-fits are `jobSchema.config` and `multiFitSchema.config`, respectively. These files specify which settings are allowed per block, and how the arguments should look like. The available blocks are: - `Job` - `Fit` - `Limit` - `Significance` - `Options` - `Region` - `Sample` - `NormFactor` - `ShapeFactor` - `Systematic` - `TruthSample` - `Unfolding` - `UnfoldingSample` - `UnfoldingSystematic`

Config include

You can use `INCLUDE` keyword to include a content of another config/text file to the config file which is being read. In order to do that, use:

```
INCLUDE: <path to the config to include>
```

where path to the config is always a relative path from the config which is being read (which contains the `INCLUDE` statement). Nested config includes are supported.

Replacement file

You can define placeholders in your config file, which are replaced with values specified in an external file, which is read at the beginning of TRExFitter execution. This requires adding an additional option into your config, as part of the Job block:

```
ReplacementFile: path/to/file.txt
```

The replacement file should have the following structure:

```
# comment
XXX_placeholder: 0.1
XXX_another_placeholder: 0.2
% also a comment
```

Note that all placeholders must start with `XXX`. In your config file, you can then refer to the placeholders like this:

```
Sample: "ttbar"
MCweight: XXX_placeholder
```

If you would like to ensure that the replacement works correctly, set your `DebugLevel` to a minimum value of 1 and check the output of the framework.

Command line options

Currently the supported options are:

Option	Effect
Job	to provide a new name for the output folder
Regions	to limit the regions to use to the list specified
Samples	to limit the samples to use to the list specified
Systematics	to limit the systematics to use to the list specified
Signal(s)	in case more than one SIGNAL sample is specified in your config file, you can specify which one you want to run on (for plots, workspace creation and fits/limits/significance); GHOST samples can be promoted to SIGNAL samples in this way, and multiple samples can be specified using <code>Samples:smp1,smp2</code>
Exclude	to exclude certain Regions / Samples / Systematics
Suffix	used for plots, workspace, fit results, etc
SaveSuffix	used for saving histograms with a suffix (to be merged / renamed later, see Input File Merging with hupdate section)
Update	if TRUE, the output .root file is updated, otherwise is overwrote
StatOnlyFit	if TRUE, the same as Fit->StatOnlyFit
StatOnly	if TRUE, no systematics nor MC stat uncertainties will be considered (equivalent to set StatOnly: TRUE in the Job block of the config), use <code>Systematics=NONE</code> instead to keep MC stat uncertainties
Ranking	see Ranking Plot Distribution section
FitResults	the specified fit results file will be used, for chi^2 and unfolding plots (instead of the file <code>jobName/Fits/jobName.txt</code>)

Option	Effect
FitResultsRootFile	the specified ROOT file with the FitResults, for instance for post-fit plots (instead of the file <code>jobName/Fits/jobName.root</code>)
NPValuesFromFitResults	Fit results <code>.txt</code> file containing values of NPs to be used in building a Asimov-data
FitType	can be set to SPLUSB or BONLY to replace the option in the config file
LumiScale	as the options in config file
BootstrapIdx	see description of Bootstrap option in config (under Job)
BootstrapSyst	see description of BootstrapSyst option in config (under Job)
BootstrapSample	see description of BootstrapSample option in config (under Job)
GroupedImpact	see Grouped Impact section
OutputDir	see Job settings section
LimitParamValue	see Limit settings section (ParamValue)
RunLHscan	Default state is true. Set to FALSE to stop the Likelihood scan from being performed.
LHscanVariable	set a NP/POI for the likelihood scan can be used for parallelization of the code
LHscanStep	Assign integer value ranging from 0 to nScanSteps-1. Used to parallel process likelihood scan points
CreateCache	Advanced feature: see Likelihood Scan
UseCache	Advanced feature: see Likelihood Scan

Option	Effect
LHscanStepY	define which step of the parallelized 2D scan should be performed (has to be an integer between 0 and LHscanStepsY-1)
FitBlind	see Fit settings section
BlindedParameters	see Fit settings section
POIASimovOverride	overrides any value set in the config file, see Fit settings section
CustomAsimovOverride	overrides the CustomAsimov set in the config file, see Job settings section
ConstNFs	Comma separated list of NormFactors to be set to constant. Their values can be optionally provided as e.g. <code>ConstNFs=mu_A@1.01,mu_B@0.94</code> (otherwise uses the default value indicated in the settings, see NormFactor)
FixNPs	Comma separated list of NPs to be set to constant in the fit (see the Fit settings section). Their values are provided as e.g. <code>FixNPs=alpha_JET_JVT@-0.45,gamma_stat_SR_bin_0@1.002</code>
CustomFunctionsExecutes	Semicolon separated list of function to execute
NumCPU	Number of CPUs used in the minimisation
RecreateBinningFiles	see Job settings section
LimitToysSuffix	Set suffix to apply to files produced in limit setting using toys. Useful when generating multiple independent parallel toys jobs with different seeds.
LimitToysSeed	see Limit settings section (ToysSeed)
FitBootstrappedData	see Fit settings section
WorkspaceCreationType	see Job settings section

Option	Effect
ToysNpValuesFile	see Fit settings section
AddNFsToPOIs	see Fit settings section
SignificanceToysSeed	see Significance settings section
LimitToysSeed	see Limit settings section

Note: the wild-card `*` is supported. For example:

```
trex-fitter n config/ttH2015.config 'Regions=HThad_ge6jge4b:Exclude=BTag_*
```

You can use command-line options to parallelise `TRExFitter` jobs that take a long time. Some discussion of this can be found in the [Distributing TRExFitter Jobs](#) section.